

# Poshi

#programming

#liferay

---

## Warning

O poshi nao esta mais sendo usado

---

O Poshi (abreviação de Portal Open Source High-level Infrastructure) é uma ferramenta de automação de testes usada dentro do ecossistema da [Liferay](#).

## O que ele é, na prática

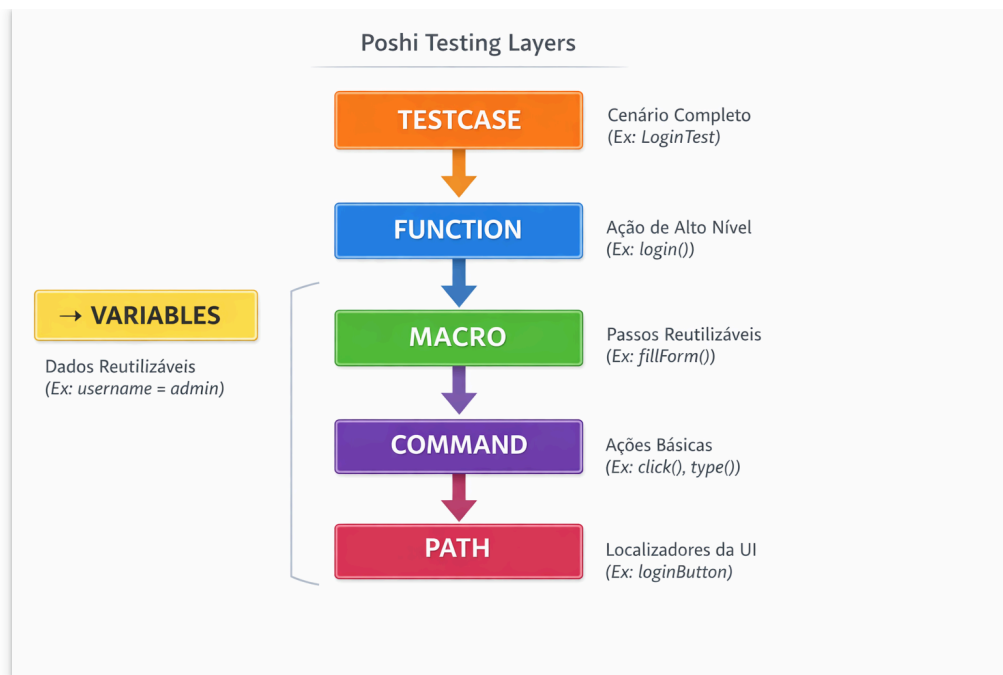
É um framework de testes funcionais automatizados, voltado principalmente para testar interfaces web (UI). Ele simula ações reais de um usuário, como:

- Clicar em botões
- Preencher formulários
- Navegar entre páginas
- Validar textos e comportamentos

## Para que serve

O objetivo principal é garantir que o sistema (como o portal da Liferay) funcione corretamente após mudanças no código.

## Layers



## 1. Testcases (camada mais alta)

São os testes em si.

- Representam um fluxo completo (ex: login, criar usuário)
- Orquestram chamadas para funções, macros, etc.

Exemplo:

```
<testcase name="LoginTest">
  <command name="login" />
  <command name="verifyDashboard" />
</testcase>
```

**Função:** definir o cenário de teste.

---

## 2. Functions

São ações reutilizáveis mais “inteligentes”.

- Podem conter lógica
- Podem chamar macros e outros comandos
- Representam uma ação completa (ex: fazer login)

Exemplo:

```
<function name="login">  
  <command name="type" target="username" value="admin" />  
  <command name="type" target="password" value="123" />  
  <command name="click" target="loginButton" />  
</function>
```

**Função:** encapsular comportamento.

---

### 3. Macros

São blocos reutilizáveis mais simples que functions.

- Normalmente não têm lógica complexa
- São mais “baixos nível”
- Reaproveitam sequências de comandos

Exemplo:

```
<macro name="clickLogin">  
  <command name="click" target="loginButton" />  
</macro>
```

**Função:** evitar repetição de comandos básicos.

---

### 4. Paths

Definem localizadores de elementos da interface (UI).

- Centralizam seletores (CSS, XPath, etc.)
- Evitam hardcode espalhado

Exemplo:

```
<path name="loginButton">  
  //button[@id='login']  
</path>
```

**Função:** desacoplar o teste da estrutura da página.

Se o HTML mudar, você altera só o path.

---

## 5. Variables

Armazenam valores reutilizáveis.

- Dados de entrada (usuário, senha, URLs)
- Constantes usadas nos testes

Exemplo:

```
<var name="username">admin</var>
```

**Função:** evitar valores fixos espalhados.

---

Diferença entre `function` e `macro`

Erro comum (importante)

Um erro frequente é usar macro como se fosse function:

- Exemplo ruim:

```
<testcase>  
  <command name="typeUsername" />  
  <command name="typePassword" />  
  <command name="clickLogin" />  
</testcase>
```

Problema:

o teste fica técnico demais  
perde legibilidade

- Correto:

```
<testcase>  
  <command name="login" />  
</testcase>
```

- “Isso representa uma intenção do usuário?” → **function**
- “Isso é só um passo técnico?” → **macro**